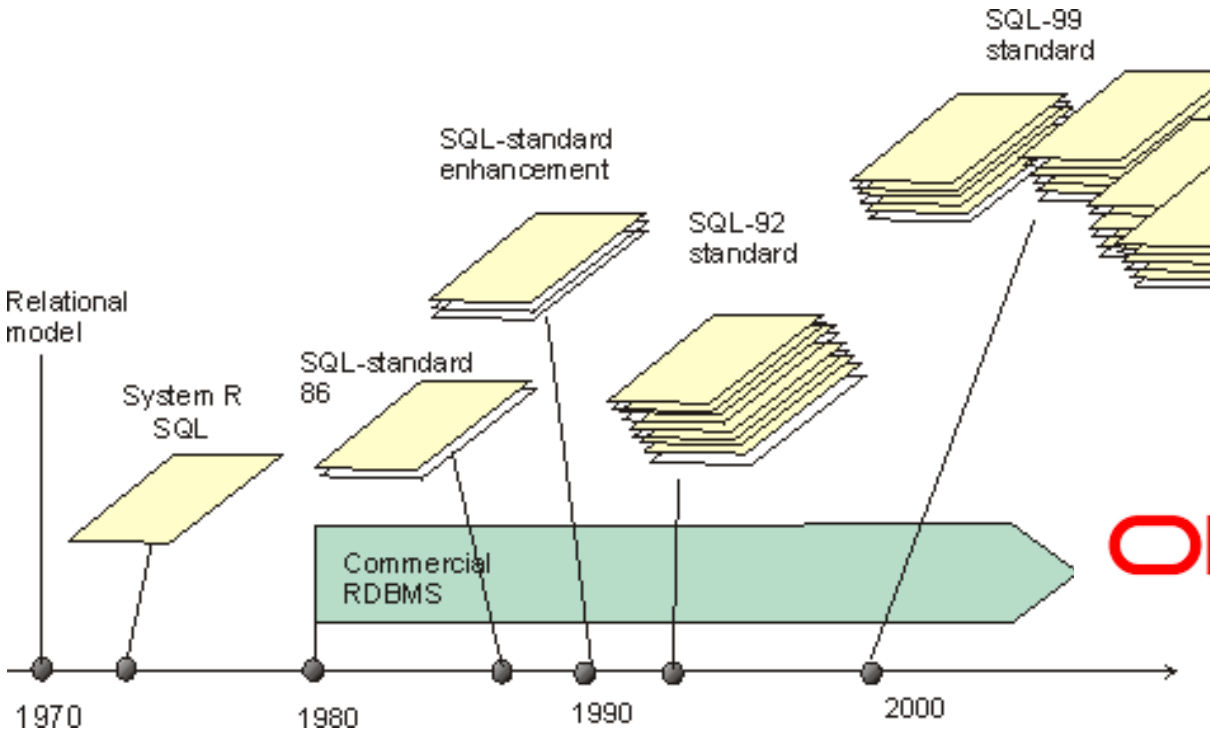
Clemson Means
BUSINESS

Chapter 4
Structural Query Language (SQL)

Instructor: He Li

Structured Query Language (SQL) –

the standard for relational database management systems (RDBMS)



Department of MANAGEMENT
Clemson University

IBM DB2

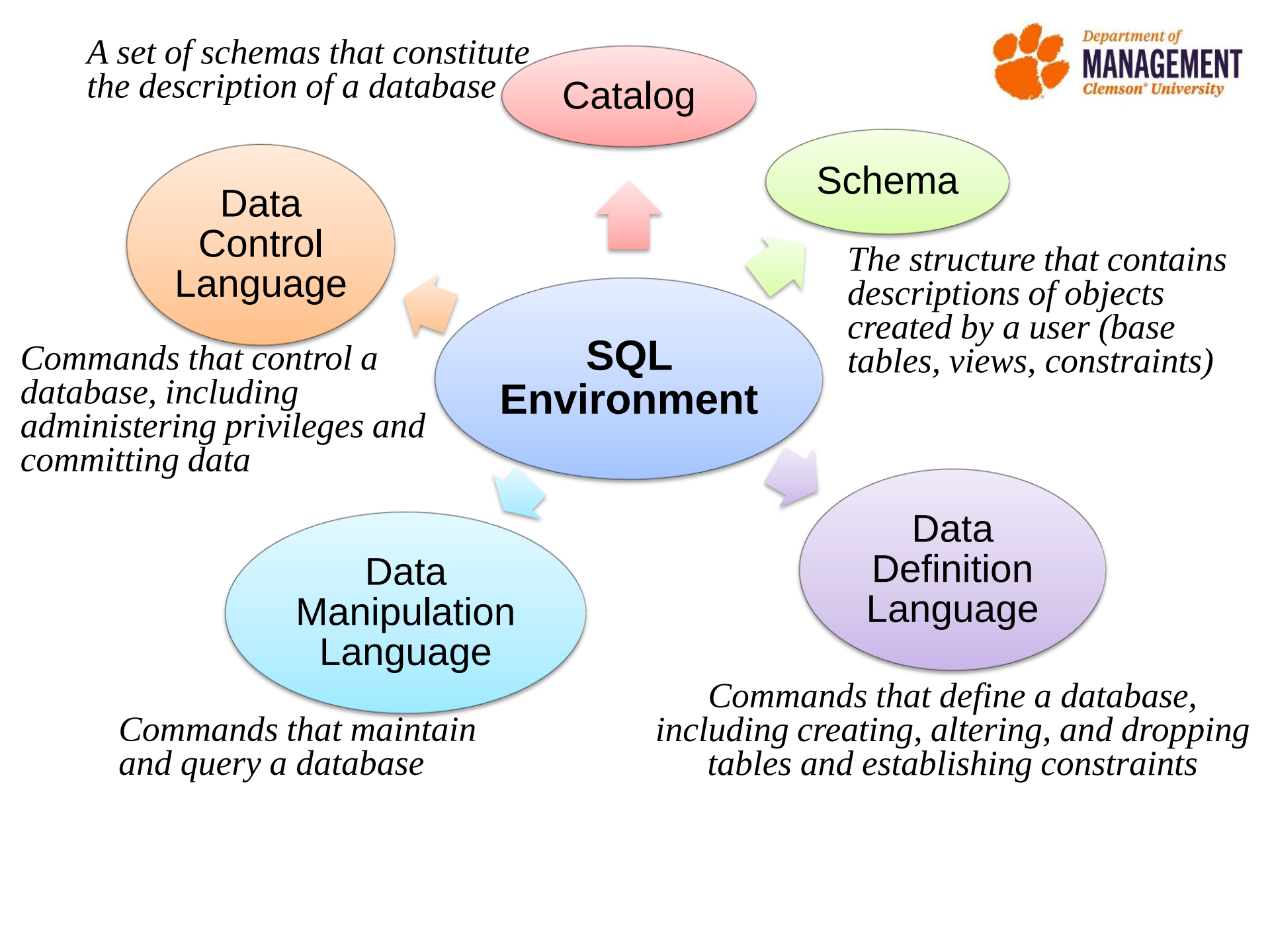
MySQL

PostgreSQL

ORACLE

TERADATA

Microsoft SQL Server


A set of schemas that constitute the description of a database
Catalog
Schema
The structure that contains descriptions of objects created by a user (base tables, views, constraints)
Data Control Language
Commands that control a database, including administering privileges and committing data
SQL Environment
Data Manipulation Language
Commands that maintain and query a database
Data Definition Language
Commands that define a database, including creating, altering, and dropping tables and establishing constraints

SQL Data Types

Department of MANAGEMENT Clemson University

String	CHARACTER (CHAR)	Stores string values containing any characters in a character set. CHAR is defined to be a fixed length.
	CHARACTER VARYING (VARCHAR or VARCHAR2)	Stores string values containing any characters in a character set but of definable variable length.
	BINARY LARGE OBJECT (BLOB)	Stores binary string values in hexadecimal format. BLOB is defined to be a variable length. (Oracle also has CLOB and NCLOB, as well as BFILE for storing unstructured data outside the database.)
Number	NUMERIC	Stores exact numbers with a defined precision and scale.
	INTEGER (INT)	Stores exact numbers with a predefined precision and scale of zero.
Temporal	TIMESTAMP	Stores a moment an event occurs, using a definable fraction-of-a-second precision.Value adjusted to the user's session time zone (available in Oracle and MySQL)
	TIMESTAMP WITH LOCAL TIME ZONE	
Boolean	BOOLEAN	Stores truth values: TRUE, FALSE, or UNKNOWN.

Illustrative Sample Data

Department of MANAGEMENT
Clemson University

Customer_T

CustomerID	CustomerName	CustomerAddress	CustomerCit	CustomerStat	CustomerPostalCod
1	Contemporary Casuals	1355 S Hines Blvd	Gainesville	FL	32601-2871
2	Value Furniture	15145 S.W. 17th St.	Plano	TX	75094-7743
3	Home Furnishings	1900 Allard Ave.	Albany	NY	12209-1125
4	Eastern Furniture	1925 Beltline Rd.	Carteret	NJ	07008-3188
5	Impressions	5585 Westcott Ct.	Sacramento	CA	94206-4056
6	Furniture Gallery	325 Flatiron Dr.	Boulder	CO	80514-4432
7	Period Furniture	394 Rainbow Dr.	Seattle	WA	97954-5589
8	California Classics	816 Peach Rd.	Santa Clara	CA	96915-7754
9	M and H Casual Furniture	37			20-2314
10	Seminole Interiors	24			46-4423
11	American Euro Lifestyles	24			08-5621
12	Battle Creek Furniture	34			15-3401
13	Heritage Furnishings	66			13-8834
14	Kaneohe Homes	11			44-2537
15	Mountain Scenes	41			03-4432

Record: 1 of 15

OrderLine_T

OrderID	ProductID	OrderedQuantity
1001	1	2
1001	2	2
1001	4	1
1002	3	5
1003	3	3
1004	6	2
1004	8	2
1005	4	4
1006	4	1
1006	5	2
1006	7	2
1007	1	3
1007	2	2
1008	3	3
1008	8	3
1009	4	2
1009	7	3
1010	8	10

Record: 1 of 18

Order_T

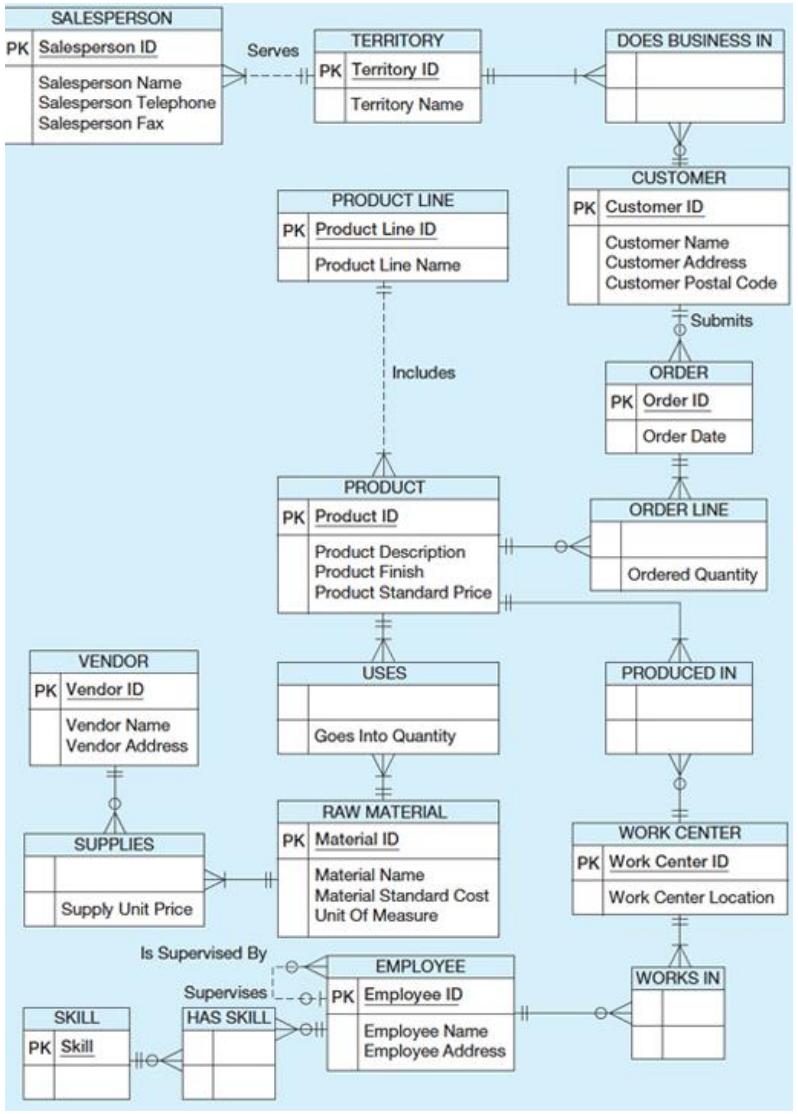
OrderID	OrderDate	CustomerID
1001	10/21/2018	1
1002	10/21/2018	8
1003	10/22/2018	15
1004	10/22/2018	5
1005	10/24/2018	3
1006	10/24/2018	2
1007	10/27/2018	11
1008	10/30/2018	12
1009	11/5/2018	4
1010	11/5/2018	1

Record: 1 of 10

Product_T

ProductID	ProductDescriptio	ProductFinisl	ProductStandardPric	ProductLineID
1	End Table	Cherry	$175.00	1
2	Coffee Table	Natural Ash	$200.00	2
3	Computer Desk	Natural Ash	$375.00	2
4	Entertainment Cente	Natural Maple	$650.00	3
5	Writers Desk	Cherry	$325.00	1
6	8-Drawer Desk	White Ash	$750.00	2
7	Dining Table	Natural Ash	$800.00	2
8	Computer Desk	Walnut	$250.00	3

Record: 9 of 9



Creating a Database in SQL

Department of MANAGEMENT Clemson University

- Syntax:

CREATE SCHEMA database_name
AUTHORIZATION owner_user id

CREATE SCHEMA	Used to define the portion of a database that a particular user owns. Schemas are dependent on a catalog and contain schema objects, including base tables and views, domains, constraints, assertions, character sets, collations, and so forth.
CREATE TABLE	Defines a new table and its columns. The table may be a base table or a derived table. Tables are dependent on a schema. Derived tables are created by executing a query that uses one or more tables or views.
CREATE VIEW	Defines a logical table from one or more tables or views. Views may not be indexed. There are limitations on updating data through a view. Where views can be updated, those changes can be transferred to the underlying base tables originally referenced to create the view.

Creating Tables: General Syntax

CREATE TABLE *tablename*
({*column definition* [*table constraint*] } . , . .
[ON COMMIT {DELETE | PRESERVE} ROWS]);

where *column definition* ::=
column_name
{*domain name* | datatype [(*size*)] }
[*column_constraint_clause*. . .]
[*default value*]
[*collate clause*]

and *table constraint* ::=
[CONSTRAINT *constraint_name*]
Constraint_type [*constraint_attributes*]

Example: SQL Database Definition Commands

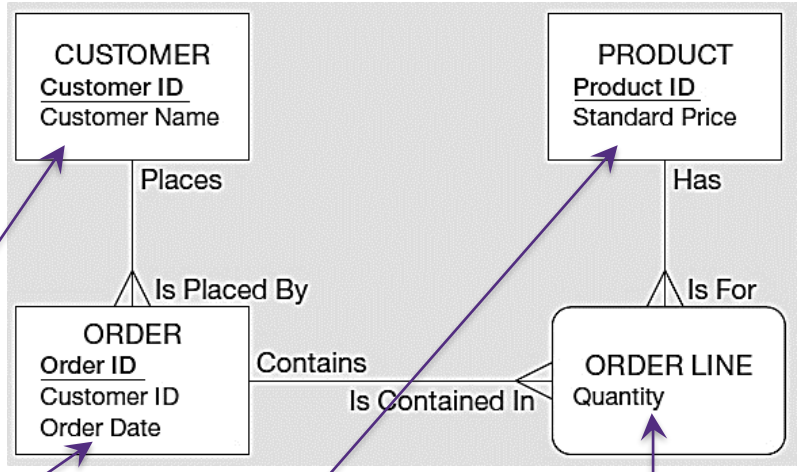
Department of MANAGEMENT Clemson University

```
CREATE TABLE Customer_T
                (CustomerID                    NUMBER(11,0)        NOT NULL,
                 CustomerName                  VARCHAR2(25)        NOT NULL,
                 CustomerAddress               VARCHAR2(30),
                 CustomerCity                  VARCHAR2(20),
                 CustomerState                 CHAR(2),
                 CustomerPostalCode            VARCHAR2(9),
CONSTRAINT Customer_PK PRIMARY KEY (CustomerID));
```

```
CREATE TABLE Order_T
                (OrderID                       NUMBER(11,0)        NOT NULL,
                 OrderDate                     DATE DEFAULT SYSDATE,
                 CustomerID                    NUMBER(11,0),
CONSTRAINT Order_PK PRIMARY KEY (OrderID),
CONSTRAINT Order_FK FOREIGN KEY (CustomerID) REFERENCES Customer_T(CustomerID));
```

```
CREATE TABLE Product_T
                (ProductID                     NUMBER(11,0)        NOT NULL,
                 ProductDescription            VARCHAR2(50),
                 ProductFinish                 VARCHAR2(20)
                        CHECK (ProductFinish IN ('Cherry', 'Natural Ash', 'White Ash',
                                                 'Red Oak', 'Natural Oak', 'Walnut')),
                 ProductStandardPrice          DECIMAL(6,2),
                 ProductLineID                 INTEGER,
CONSTRAINT Product_PK PRIMARY KEY (ProductID));
```

```
CREATE TABLE OrderLine_T
                (OrderID                       NUMBER(11,0)        NOT NULL,
                 ProductID                     INTEGER             NOT NULL,
                 OrderedQuantity               NUMBER(11,0),
CONSTRAINT OrderLine_PK PRIMARY KEY (OrderID, ProductID),
CONSTRAINT OrderLine_FK1 FOREIGN KEY (OrderID) REFERENCES Order_T(OrderID),
CONSTRAINT OrderLine_FK2 FOREIGN KEY (ProductID) REFERENCES Product_T(ProductID));
```



Step 1. Defining Attributes and Their Data Types

Department of MANAGEMENT Clemson University

```
CREATE TABLE Product_T
        (ProductID                  NUMBER(11,0)     NOT NULL,
         ProductDescription         VARCHAR2(50),
         ProductFinish              VARCHAR2(20)
                  CHECK (ProductFinish IN ('Cherry', 'Natural Ash', 'White Ash',
                                           'Red Oak', 'Natural Oak', 'Walnut')),
         ProductStandardPrice       DECIMAL(6,2),
         ProductLineID              INTEGER,
CONSTRAINT Product_PK PRIMARY KEY (ProductID));
```

Step 2. Non-Nullable Specifications

- Some primary keys are composite – composed of multiple attributes

```
CREATE TABLE OrderLine_T
        (OrderID              NUMBER(11,0)    NOT NULL,
         ProductID            INTEGER         NOT NULL,
         OrderedQuantity      NUMBER(11,0),
CONSTRAINT OrderLine_PK PRIMARY KEY (OrderID, ProductID),
CONSTRAINT OrderLine_FK1 FOREIGN KEY (OrderID) REFERENCES Order_T(OrderID),
CONSTRAINT OrderLine_FK2 FOREIGN KEY (ProductID) REFERENCES Product_T(ProductID));
```

Step 3. Controlling the Values in Attributes

Department of MANAGEMENT Clemson University

```
CREATE TABLE Order_T
                (OrderID                        NUMBER(11,0)        NOT NULL,
                 OrderDate                      DATE DEFAULT SYSDATE,
                 CustomerID                     NUMBER(11,0),
CONSTRAINT Order_PK PRIMARY KEY (OrderID),
CONSTRAINT Order_FK FOREIGN KEY (CustomerID) REFERENCES Customer_T(CustomerID));

CREATE TABLE Product_T
                (ProductID                      NUMBER(11,0)        NOT NULL,
                 ProductDescription             VARCHAR2(50),
                 ProductFinish                  VARCHAR2(20)
                        CHECK (ProductFinish IN ('Cherry', 'Natural Ash', 'White Ash',
                                                 'Red Oak', 'Natural Oak', 'Walnut')),
                 ProductStandardPrice           DECIMAL(6,2),
                 ProductLineID                  INTEGER,
CONSTRAINT Product_PK PRIMARY KEY (ProductID));
```

Step 4. Identifying Foreign Keys and Establishing Relationships

Department of MANAGEMENT Clemson University

```
CREATE TABLE Customer_T
                (CustomerID                NUMBER(11,0)        NOT NULL,
                 CustomerName              VARCHAR2(25)        NOT NULL,
                 CustomerAddress           VARCHAR2(30),
                 CustomerCity              VARCHAR2(20),
                 CustomerState             CHAR(2),
                 CustomerPostalCode        VARCHAR2(9),
CONSTRAINT Customer_PK PRIMARY KEY (CustomerID));

CREATE TABLE Order_T
                (OrderID                   NUMBER(11,0)        NOT NULL,
                 OrderDate                 DATE DEFAULT SYSDATE,
                 CustomerID                NUMBER(11,0),
CONSTRAINT Order_PK PRIMARY KEY (OrderID),
CONSTRAINT Order_FK FOREIGN KEY (CustomerID) REFERENCES Customer_T(CustomerID));
```

Primary key of parent table

Data Integrity Controls

- **Referential integrity** – constraint that ensures that foreign key values of a table must match primary key values of a related table in a 1:N relationship

- **Restricting:**
  - Deletes of primary records
  - Updates of primary records
  - Inserts of dependent records

- Relational integrity is enforced via the primary-key to foreign-key match

Restricted Update: A customer ID can only be deleted if it is not found in ORDER table.

```
CREATE TABLE CustomerT
            (CustomerID          INTEGER DEFAULT '999'    NOT NULL,
             CustomerName        VARCHAR(40)              NOT NULL,
             . . .
CONSTRAINT Customer_PK PRIMARY KEY (CustomerID),
ON UPDATE RESTRICT);
```

Cascaded Update: Changing a customer ID in the CUSTOMER table will result in that value changing in the ORDER table to match.

```
. . . ON UPDATE CASCADE);
```

Set Null Update: When a customer ID is changed, any customer ID in the ORDER table that matches the old customer ID is set to NULL.

```
. . . ON UPDATE SET NULL);
```

Set Default Update: When a customer ID is changed, any customer ID in the ORDER tables that matches the old customer ID is set to a predefined default value.

```
. . . ON UPDATE SET DEFAULT);
```

Changing Tables

- **ALTER TABLE** statement allows you to change column specifications:

```
ALTER TABLE table_name alter_table_action;
```

  - alter_table_action could be:

```
ADD [COLUMN] column_definition
ALTER [COLUMN] column_name SET DEFAULT default-value
ALTER [COLUMN] column_name DROP DEFAULT
DROP [COLUMN] column_name [RESTRICT] [CASCADE]
ADD table_constraint
```

  - example (adding a new column with a default value):

```
ALTER TABLE CUSTOMER_T ADD COLUMN
CustomerType VARCHAR2 (10) DEFAULT
“Commercial”;
```

- **DROP TABLE** statement allows you to remove tables from your schema:

```
DROP TABLE CUSTOMER_T
```

Inserting Data

Department of MANAGEMENT Clemson University

- Adds one or more rows to a table
- Inserting into a table:

```
INSERT INTO Customer_T VALUES (001, 'Contemporary
Casuals', '1355 S. Himes Blvd.', 'Gainesville', 'FL', 32601);
```

- Inserting a record that has some null attributes requires identifying the fields that actually get data:

```
INSERT INTO Product_T (ProductID, ProductDescription,
ProductFinish, ProductStandardPrice) VALUES (1, 'End
Table', 'Cherry', 175, 8);
```

- Inserting from another table:

```
INSERT INTO CaCustomer_T SELECT * FROM
Customer_T WHERE CustomerState = 'CA';
```

Deleting Data

- Removes rows from a table
- Delete certain rows

```
DELETE FROM CUSTOMER_T WHERE
CUSTOMERSTATE = 'HI';
```

- Delete all rows

```
DELETE FROM CUSTOMER_T;
```

Updating and Merging Data

- **Updating Data:** Modifies data in existing rows

```
UPDATE Product_T SET
ProductStandardPrice = 775 WHERE
ProductID = 7;
```

- **Merging Data:** Makes it easier to update a table. It allows combination of Insert and Update in one statement. It is useful for updating master tables with new data.

```
MERGE INTO Product_T AS PROD
USING
(SELECT ProductID, ProductDescription, ProductFinish,
ProductStandardPrice, ProductLineID FROM Purchases_T) AS PURCH
      ON (PROD.ProductID = PURCH.ProductID)
WHEN MATCHED THEN UPDATE
      PROD.ProductStandardPrice = PURCH.ProductStandardPrice
WHEN NOT MATCHED THEN INSERT
      (ProductID, ProductDescription, ProductFinish, ProductStandardPrice,
      ProductLineID)
      VALUES(PURCH.ProductID, PURCH.ProductDescription,
      PURCH.ProductFinish, PURCH.ProductStandardPrice,
        PURCH.ProductLineID);
```

Queries for Single Tables

Department of MANAGEMENT Clemson University

- Clauses of the **SELECT** statement:
  - SELECT: List the columns (and expressions) to be returned from the query
  - FROM: Indicate the table(s) or view(s) from which data will be obtained
  - WHERE: Indicate the conditions under which a row will be included in the result
  - GROUP BY: Indicate categorization of results
  - HAVING: Indicate the conditions under which a category (group) will be included
  - ORDER BY: Sorts the result according to specified criteria

```
SELECT [ALL | DISTINCT]
  {* | TABLE.* | expression [AS alias] [, expression [AS Alias] ...}
    FROM tablename [alias] [, tablename [alias] ...
      [WHERE condition]
      [GROUP BY expression [, expression] ... ] [HAVING condition]
[ { UNION [ALL] | INTERSECT | MINUS } SELECT ... ]
      [ORDER BY {expression | position} [ASC | DESC] [, expression | position
[ASC | DESC ] ...
```

Basic SELECT Statement

- **Q1:** List all the data about customers

```
SELECT *
FROM "db_pvfc11_big"."Customer_T";
```

CUSTOME...	CUSTOME...	CUSTOME...	CUSTOME...	CUSTOME...	CUSTOME...
16	ABC Furniture C	152 Geramino Dı	Rome	NY	13440
3	Home Furnishing	1900 Allard Ave	Albany	NY	12209-1125
12	Flanigan Furnitur	Snow Flake Rd	Ft Walton Beach	FL	32548
9	A Carpet	434 Abe Dr	Rome	NY	13440
5	Impressions	5585 Westcott C	Sacramento	CA	94206-4056
14	Wild Bills	Four Horse Rd	Oak Brook	Il	60522
8	Dunkins Furniture	7700 Main St	Syracuse	NY	31590
2	Value Furnitures	15145 S.W. 17th	Plano	TX	75094-7734
13	Ikards	1011 S. Main St	Las Cruces	NM	88001
6	Furniture Gallery	325 Flatiron Dr.	Boulder	CO	80514-4432
1	Contemporary C	1355 S Hines Bl	Gainesville	FL	32601-2871
15	Janets Collection	Janet Lane	Virginia Beach	VA	10012
4	Eastern Furniture	1925 Beltline Rd.	Carteret	NJ	07008-3188
7	New Furniture	Palace Ave	Farmington	NM	

14 rows total

Basic SELECT Statement

- **Q2:** List name and post code for all customers

```
SELECT CustomerName, CustomerPostalCode
FROM "db_pvfc11_big"."Customer_T";
```

CUSTOME…	CUSTOME…
ABC Furniture C	13440
Home Furnishing	12209-1125
Flanigan Furnitur	32548
A Carpet	13440
Impressions	94206-4056
Wild Bills	60522
Dunkins Furniture	31590
Value Furnitures	75094-7734
Ikards	88001
Furniture Gallery	80514-4432
Contemporary C	32601-2871
Janets Collection	10012
Eastern Furniture	07008-3188
New Furniture	

14 rows total

Using Comparison Operators

- **Q3:** List the name and address of all customers in postal code 32601-2871

```
SELECT CustomerName, CustomerAddress
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerPostalCode = '32601-2871';
```

CUSTOMERNAME	CUSTOMERAD…
Contemporary Casual	1355 S Hines Blvd

Comparison operators include:

- **= Equal to**
- **> Greater than**
- **>= Greater than or equal to**
- **< Less than**
- **<= Less than or equal to**
- **<> Not equal to !=**

Using Comparison Operators

- **Q4:** Which products have a standard price of? Show the product description and standard price less than $275

```
SELECT ProductDescription, ProductStandardPrice
FROM "db_pvfc11_big"."Product_T"
WHERE ProductStandardPrice < 275;
```

PRODUCTDESCRIPTION	PRODUCTSTANDARDPRICE
	0.0000
96" Bookcase	225.0000
Pine End Table	256.0000
48" Bookcase	175.0000
Birch Coffee Table	200.0000
	0.0000
96" Bookcase	200.0000
Nightstand	150.0000
Cherry End Table	175.0000
48" Bookcase	150.0000

10 rows total

Using Comparison Operators

- **Q5:** List OrderID, CustomerID, and OrderDate for all orders ordered since August 1, 2010.

```
SELECT OrderID, CustomerID, OrderDate
FROM "db_pvfc11_big"."Order_T"
WHERE OrderDate > '2010-08-01';
```

ORDERID	CUSTOME...	ORDERDATE
71	4	10/09/08
73	12	10/09/08
76	4	10/09/15
75	1	10/09/08

4 rows total

Using Comparison Operators

- **Q6:** List ProductDescription and ProductFinish of products that isn't made of cherry.

```
SELECT ProductDescription, ProductFinish
FROM "db_pvfc11_big"."Product_T"
WHERE ProductFinish <> 'Cherry'; !=
```

PRODUCTDESCRIPTION	PRODUCTFINISH
Oak Computer Desk	Oak
8-Drawer Dresser	Oak
Amoire	Walnut
7 Grandfather Clock	Oak
96" Bookcase	Walnut
Writers Desk	Oak
Pine End Table	Pine
Writers Desk	Birch
48" Bookcase	Oak
Birch Coffee Table	Birch
High Back Leather Chair	Leather
4-Drawer Dresser	Oak
96" Bookcase	Oak
6 Grandfather Clock	Oak
8 Drawer Dresser	Birch

16 rows total

Using Expressions

- **Q7:** What are the standard price and standard price if increased by 10 percent for every product?

```
SELECT ProductID, ProductStandardPrice,
ProductStandardPrice*1.1 AS Plus10Percent
FROM "db_pvfc11_big"."Product_T";
```

PRODUCTID	PRODUCTS...	PLUS10PE...
24	0.0000	0.00000
3	750.0000	825.00000
12	800.0000	880.00000
20	1200.0000	1320.00000
19	1100.0000	1210.00000
9	225.0000	247.50000
5	325.0000	357.50000
21	256.0000	281.60000
14	300.0000	330.00000
8	175.0000	192.50000
2	200.0000	220.00000
25	0.0000	0.00000
17	362.0000	398.20000
11	500.0000	550.00000
10	200.0000	220.00000

21 rows total

Using Functions

Department of MANAGEMENT Clemson University

- **Common SQL functions:**

Mathematical	MIN, MAX, COUNT, SUM, ROUND (to round up a number to a specific number of decimal places), TRUNC (to truncate insignificant digits), and MOD (for modular arithmetic)
String	LOWER (to change to all lower case), UPPER (to change to all capital letters), INITCAP (to change to only an initial capital letter), CONCAT (to concatenate), SUBSTR (to isolate certain character positions), and COALESCE (finding the first not NULL values in a list of columns)
Date	NEXT_DAY (to compute the next date in sequence), ADD_MONTHS (to compute a date a given number of months before or after a given date), and MONTHS_BETWEEN (to compute the number of months between specified dates)
Analytical	TOP (find the top n values in a set, e.g., the top 5 customers by total annual sales)

Using Expressions

- **Q8:** What is the average standard price for all products in inventory?

```
SELECT AVG(ProductStandardPrice) AS AveragePrice
FROM "db_pvfc11_big"."Product_T";
```

AVERAGEPRICE
483.7

- **Q9:** How many different items were ordered on order number 32?

```
SELECT COUNT(*) → all raws including missing values
FROM "db_pvfc11_big"."OrderLine_T"
WHERE OrderID = 32;
```

```
COUNT(OrderID)
→ non-missing raws
```

COUNT(*)	
2	

Using Null Values

- **Q10:** Display all customers for whom we do not know their postal code.

```
SELECT *
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerPostalCode IS NULL;
```

- **Q11:** Display all customers for whom we know their postal code.

```
SELECT *
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerPostalCode IS NOT NULL;
```

Using Wildcards

- **Q12:** List CustomerName and CustomerCity for customers whose name includes 'Furniture'

```
SELECT CustomerName, CustomerCity
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerName LIKE '%Furniture%';
```

CUSTOMERNAME	CUSTOMERCITY
ABC Furniture Co.	Rome
Flanigan Furniture	Ft Walton Beach
New Furniture	Farmington
Dunkins Furniture	Syracuse
Value Furnitures	Plano
Furniture Gallery	Boulder
Eastern Furniture	Carteret

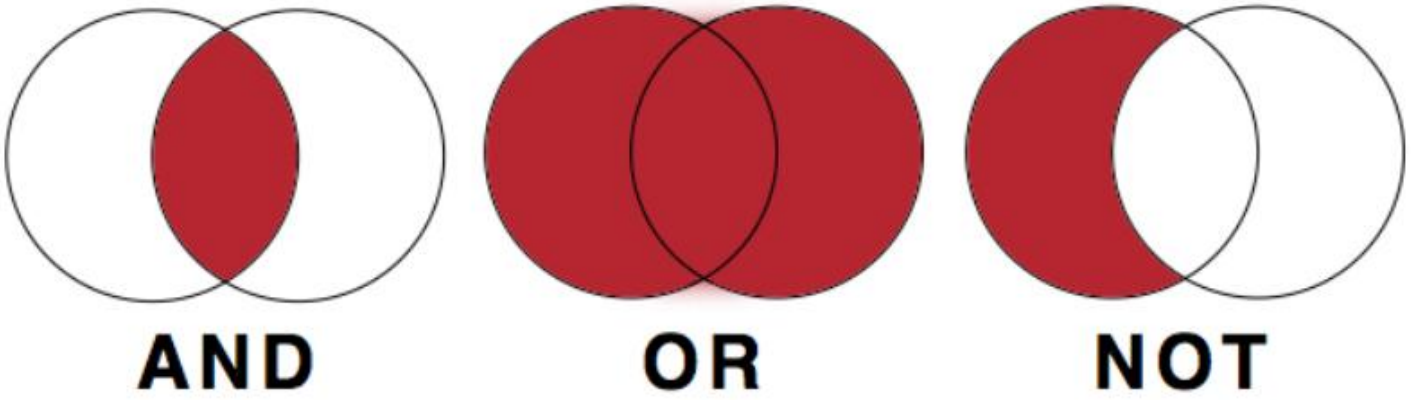
% any collection of characters

_ exactly one character

Using Boolean Operators

Department of MANAGEMENT Clemson University

AND	Joins two or more conditions and returns results only when all conditions are true.
OR	Joins two or more conditions and returns results when any conditions are true.
NOT	Negates an expression.



Using Boolean Operators

- **Q13:** List product name, finish, and standard price for all desks and all tables that costs more than $100 in the Product table.

```
SELECT ProductDescription, ProductFinish, ProductStandardPrice
FROM "db_pvfc11_big"."Product_T"
WHERE (ProductDescription LIKE '%Desk'
  OR ProductDescription LIKE '%Table')
  AND ProductStandardPrice > 100;
```

PRODUCTDESCR…	PRODUCTF…	PRODUCTS…
Oak Computer Desk	Oak	750.0000
Writers Desk	Oak	325.0000
Pine End Table	Pine	256.0000
Writers Desk	Birch	300.0000
Birch Coffee Table	Birch	200.0000
Cherry End Table	Cherry	175.0000

Using Ranges for Qualification

- **Q14:** List OrderID, CustomerID, and OrderDate for all orders ordered during the first 10 days of March, 2010.

```
SELECT OrderID, CustomerID, OrderDate
FROM "db_pvfc11_big"."Order_T"
WHERE OrderDate BETWEEN '2010-03-01' AND '2010-03-10';
```

ORDERID	CUSTOME...	ORDERDATE
24	1	10/03/10
27	4	10/03/10
26	4	10/03/10
23	4	10/03/06
20	4	10/03/06
19	4	10/03/05
21	4	10/03/06
22	4	10/03/06
25	4	10/03/10
28	4	10/03/10

Using Ranges for Qualification

- **Q15:** Which products in the Product table have a standard price between $200 and $300.

```
SELECT ProductDescription, ProductStandardPrice
FROM "db_pvfc11_big"."Product_T"
WHERE ProductStandardPrice BETWEEN 200 AND 300;
```

PRODUCTDESCRIPTION	PRODUCTSTANDARDPRICE
96" Bookcase	225.0000
Pine End Table	256.0000
Writers Desk	300.0000
Birch Coffee Table	200.0000
96" Bookcase	200.0000

Using Distinct Values

- **Q16:** What unique order numbers are included in the OrderLine table?

```
SELECT DISTINCT OrderID
FROM "db_pvfc11_big"."OrderLine_T";
```

ORDERID
24
71
54
3
55
32
49
26
48
66
65
69
56
63

26 rows total

Using Distinct Values

- **Q17:** What are the unique combinations of order number and order quantity included in the OrderLine table?

```
SELECT DISTINCT OrderID, OrderedQuantity
FROM "db_pvfc11_big"."OrderLine_T";
```

ORDERID	ORDEREDQUANTITY
4	3
69	4
26	5
2	2
1	9
5	1
51	1
3	2
56	1
1	2
54	2
65	0
49	1
52	1

36 rows total

Using IN and NOT IN with Lists

- **Q18:** List all customers who live in warmer states (i.e., FL, TX, CA).

```
SELECT *
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerState IN ('FL', 'TX', 'CA');
```

CUSTOME…	CUSTOME…	CUSTOME…	CUSTOME…	CUSTOME…	CUSTOME…
12	Flanigan Furnitur	Snow Flake Rd	Ft Walton Beach	FL	32548
5	Impressions	5585 Westcott Ct	Sacramento	CA	94206-4056
2	Value Furnitures	15145 S.W. 17th	Plano	TX	75094-7734
1	Contemporary Ca	1355 S Hines Blv	Gainesville	FL	32601-2871

4 rows total

Sorting Results Using ORDER BY

- **Q19:** List all customers who live in warmer states (i.e., FL, TX, CA). List customers alphabetically by state and alphabetically by customer within each state.

```
SELECT *
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerState IN ('FL', 'TX', 'CA')
ORDER BY CustomerState, CustomerName DESC;
```

CUSTOME...	CUSTOME...	CUSTOME...	CUSTOME...	CUSTOME...	CUSTOME...
5	Impressions	5585 Westcott Ct	Sacramento	CA	94206-4056
1	Contemporary Ca	1355 S Hines Blv	Gainesville	FL	32601-2871
12	Flanigan Furnitur	Snow Flake Rd	Ft Walton Beach	FL	32548
2	Value Furnitures	15145 S.W. 17th	Plano	TX	75094-7734

Categorizing Results Using GROUP BY

- **Q20:** Count the number of customers with addresses in each state to which we ship.

```
SELECT CustomerState, COUNT(CustomerState)
FROM "db_pvfc11_big"."Customer_T"
GROUP BY CustomerState;
```

CUSTOME…	COUNT(CU…
CA	1
CO	1
IL	1
NM	2
TX	1
VA	1
NY	4
NJ	1
FL	2

Categorizing Results Using GROUP BY

Department of MANAGEMENT Clemson University

- **Q21:** How many products are there of each finish?

```
SELECT ProductFinish, COUNT(ProductFinish)
FROM "db_pvfc11_big"."Product_T"
GROUP BY ProductFinish;
```

PRODUCTFINISH	COUNT(PRODUCTFINISH)
Walnut	3
Birch	3
Pine	1
Leather	1
Oak	8
Cherry	3
	0

Qualifying Results Using HAVING

- The HAVING clause acts like a WHERE clause, but it identifies groups, rather than rows, that meet a criterion.

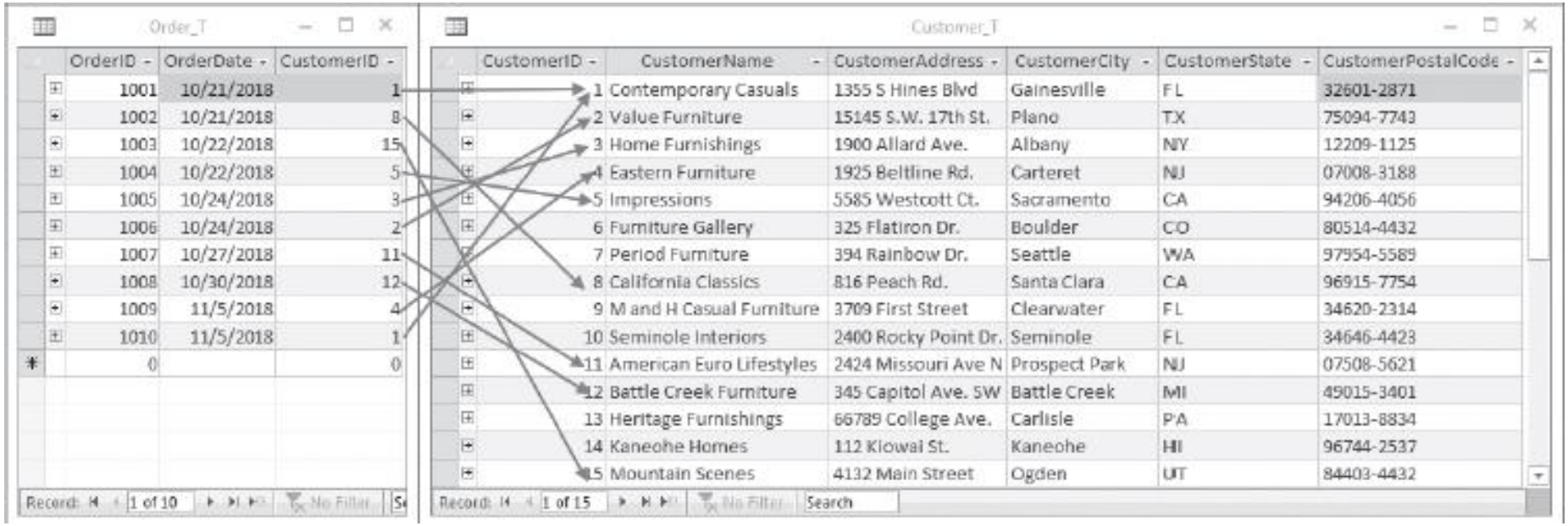
- **Q22:** Find only states with more than one customer.

```
SELECT CustomerState, COUNT(*)
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerState IN ('NM', 'FL')
GROUP BY CustomerState
HAVING COUNT(*) > 1;
```

CUSTOMERSTATE	TOTALNUMBER
NM	2
NY	4
FL	2

PART II: Queries for Multiple Tables

- **Join:** A relational operation that causes two or more tables with a common domain to be combined into a single table or view



Order_T

OrderID	OrderDate	CustomerID
1001	10/21/2018	1
1002	10/21/2018	8
1003	10/22/2018	15
1004	10/22/2018	5
1005	10/24/2018	3
1006	10/24/2018	2
1007	10/27/2018	11
1008	10/30/2018	12
1009	11/5/2018	4
1010	11/5/2018	1
0		0

Record: 1 of 10

Customer_T

CustomerID	CustomerName	CustomerAddress	CustomerCity	CustomerState	CustomerPostalCode
1	Contemporary Casuals	1355 S Hines Blvd	Gainesville	FL	32601-2871
2	Value Furniture	15145 S.W. 17th St.	Plano	TX	75094-7743
3	Home Furnishings	1900 Allard Ave.	Albany	NY	12209-1125
4	Eastern Furniture	1925 Beltline Rd.	Carteret	NJ	07008-3188
5	Impressions	5585 Westcott Ct.	Sacramento	CA	94206-4056
6	Furniture Gallery	325 Flatiron Dr.	Boulder	CO	80514-4432
7	Period Furniture	394 Rainbow Dr.	Seattle	WA	97954-5589
8	California Classics	816 Peach Rd.	Santa Clara	CA	96915-7754
9	M and H Casual Furniture	3709 First Street	Clearwater	FL	34620-2314
10	Seminole Interiors	2400 Rocky Point Dr.	Seminole	FL	34646-4423
11	American Euro Lifestyles	2424 Missouri Ave N	Prospect Park	NJ	07508-5621
12	Battle Creek Furniture	345 Capitol Ave. SW	Battle Creek	MI	49015-3401
13	Heritage Furnishings	66789 College Ave.	Carlisle	PA	17013-8834
14	Kaneohe Homes	112 Kiowai St.	Kaneohe	HI	96744-2537
15	Mountain Scenes	4132 Main Street	Ogden	UT	84403-4432

Record: 1 of 15 Search

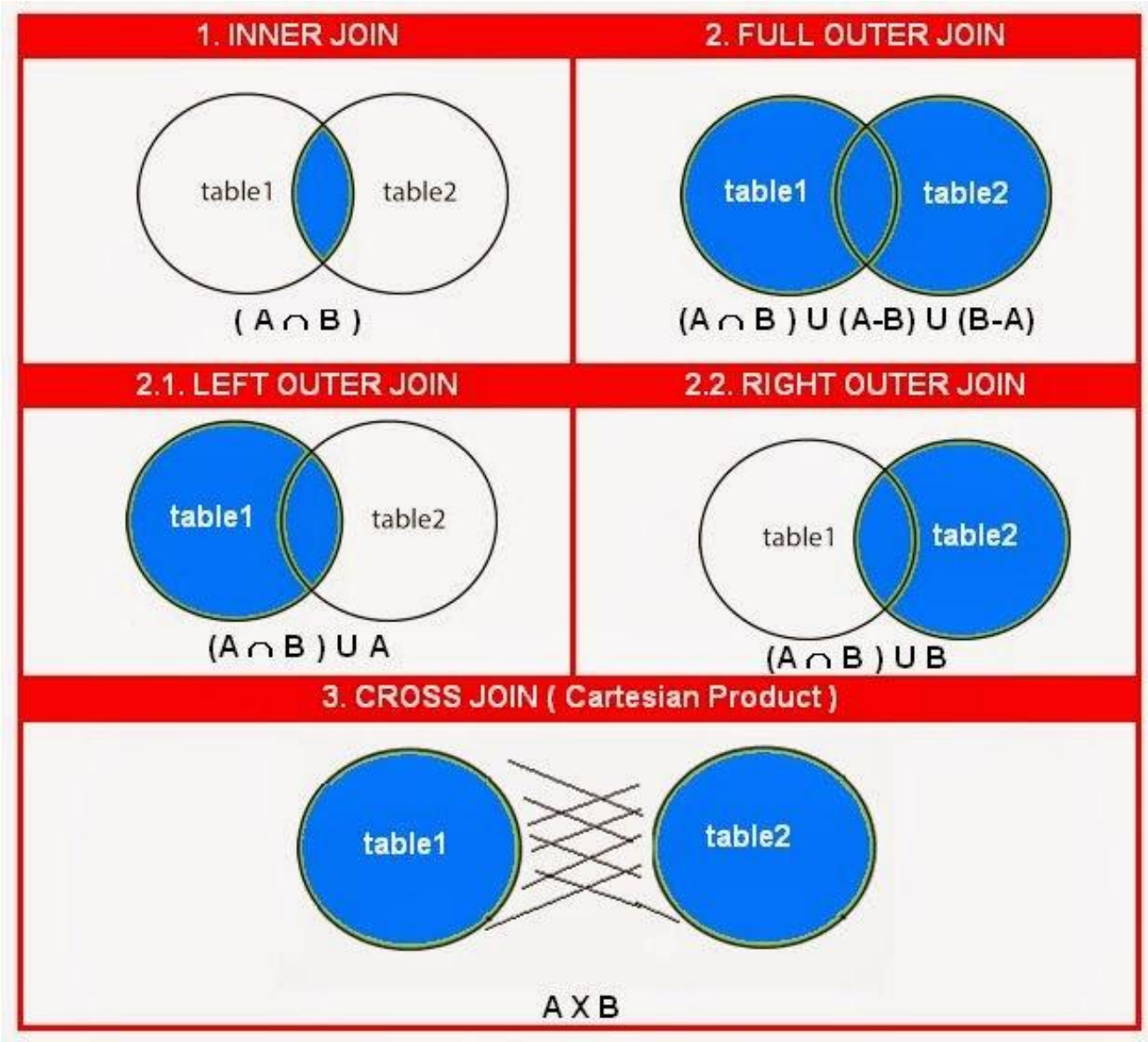
Types of Joins

Syntax: specify as **FROM** clause

table 1 explicit **JOIN** *table 2* **ON** joining condition

explicit: could be INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, or CROSS

With cross join, no need to specify condition



Inner Join

- **Q23:** What are the customer IDs and names of all customers, along with the order IDs for all the orders they have placed?

```
SELECT Customer_T.CustomerID, CustomerName, OrderID
FROM "db_pvfc11_big"."Customer_T" INNER JOIN Order_T
on Customer_T.CustomerID = Order_T.CustomerID;
```

```
SELECT Customer_T.CustomerID, CustomerName, OrderID
FROM "db_pvfc11_big"."Customer_T", Order_T
WHERE Customer_T.CustomerID = Order_T.CustomerID;
```

CUSTOME...	CUSTOME...	ORDERID
16	ABC Furniture C	51
3	Home Furnishing	2
12	Flanigan Furnitur	47
9	A Carpet	66
14	Wild Bills	58
8	Dunkins Furnitur	35
13	Ikards	59
6	Furniture Gallery	9
1	Contemporary C	46

Left Outer Join

- **Q24:** List customer name, identification number, and order number for all customers listed in the customer table. Include the customer identification number and name even if there is no order available for that customer.

```
SELECT Customer_T.CustomerID, CustomerName, OrderID
FROM "db_pvfc11_big"."Customer_T" LEFT OUTER JOIN Order_T
on Customer_T.CustomerID = Order_T.CustomerID;
```

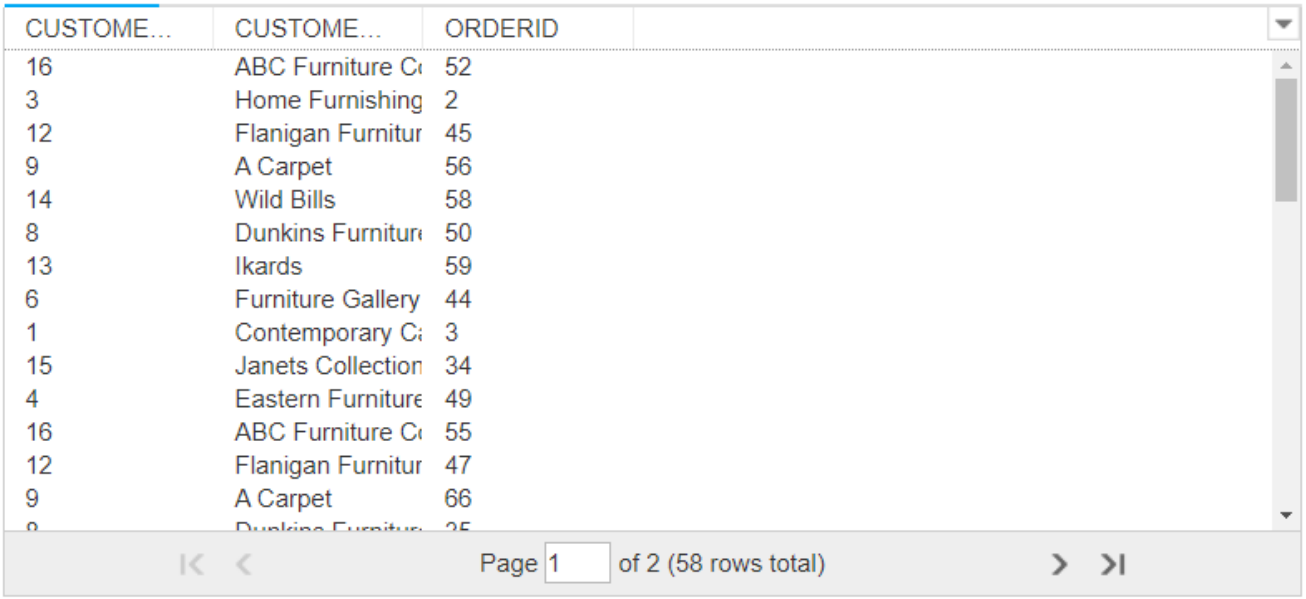
CUSTOME...	CUSTOME...	ORDERID
16	ABC Furniture C	53
3	Home Furnishing	2
12	Flanigan Furnitur	45
9	A Carpet	66
5	Impressions	
14	Wild Bills	58
8	Dunkins Furniture	35
2	Value Furnitures	
13	Ikards	59
6	Furniture Gallery	44
1	Contemporary C	48
15	Janets Collection	34
4	Eastern Furniture	30
16	ABC Furniture C	55
12	Flanigan Furnitur	41

Page 1 of 2 (61 rows total)

Right Outer Join

- **Q25:** List customer name, identification number, and order number for all customers listed in the customer table. Include the order number, even if there is no customer name and identification number available.

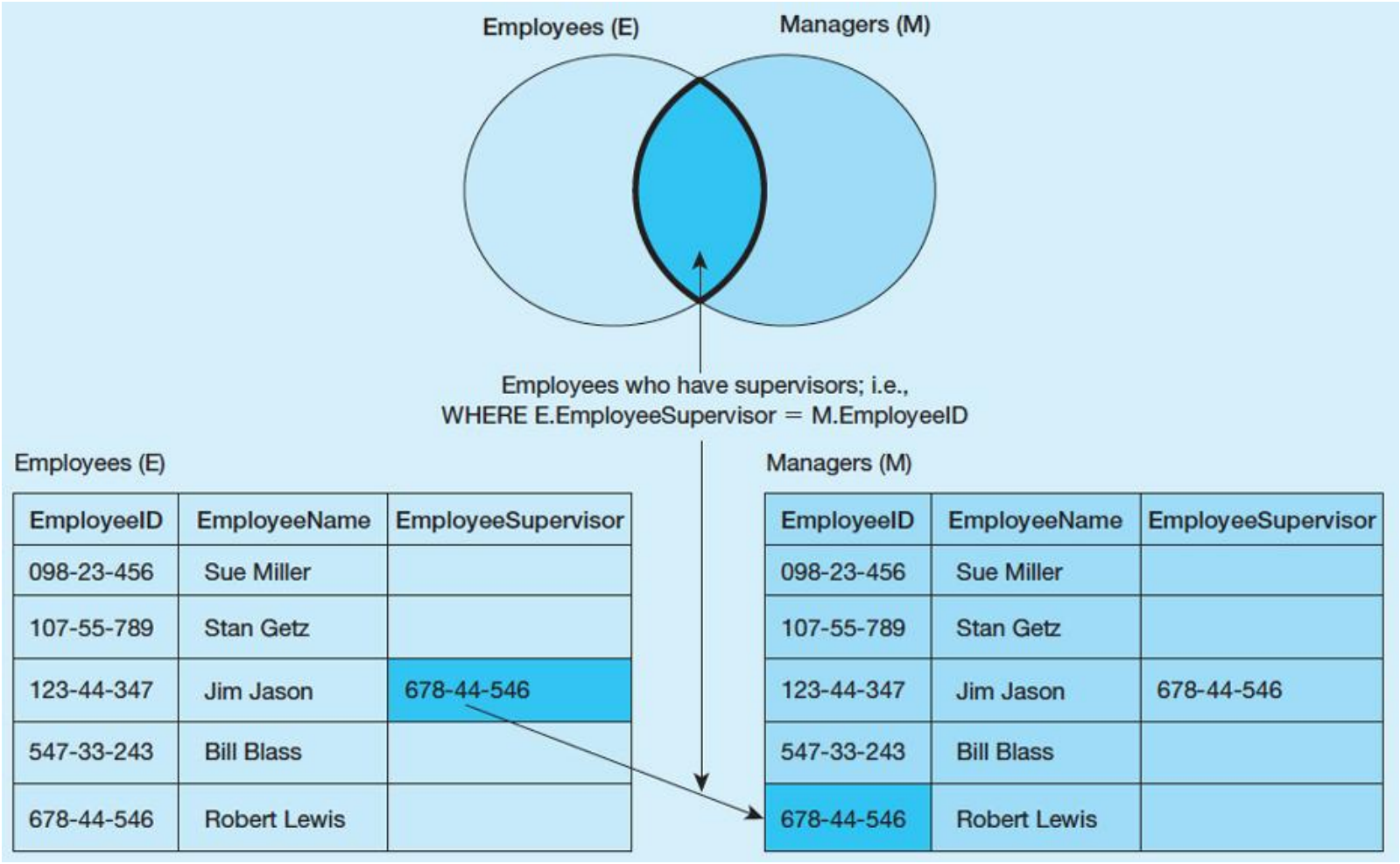
```
SELECT Customer_T.CustomerID, CustomerName, OrderID
FROM "db_pvfc11_big"."Customer_T" RIGHT OUTER JOIN Order_T
on Customer_T.CustomerID = Order_T.CustomerID;
```



CUSTOME...	CUSTOME...	ORDERID
16	ABC Furniture C	52
3	Home Furnishing	2
12	Flanigan Furnitur	45
9	A Carpet	56
14	Wild Bills	58
8	Dunkins Furnitur	50
13	Ikards	59
6	Furniture Gallery	44
1	Contemporary C	3
15	Janets Collectior	34
4	Eastern Furniture	49
16	ABC Furniture C	55
12	Flanigan Furnitur	47
9	A Carpet	66

Self-Join

- A join requires matching rows in a table with other rows in that same table, e.g., unary relationship.



Employees (E)

EmployeeID	EmployeeName	EmployeeSupervisor
098-23-456	Sue Miller	
107-55-789	Stan Getz	
123-44-347	Jim Jason	678-44-546
547-33-243	Bill Blass	
678-44-546	Robert Lewis	

Managers (M)

EmployeeID	EmployeeName	EmployeeSupervisor
098-23-456	Sue Miller	
107-55-789	Stan Getz	
123-44-347	Jim Jason	678-44-546
547-33-243	Bill Blass	
678-44-546	Robert Lewis	

Self-Join

- **Q26:** What are the employee ID and name of each employee and the name of his or her supervisor?

```
SELECT E.EmployeeID, E.EmployeeName, M.EmployeeName AS Manager
FROM "db_pvfc11_big"."Employee_T" AS E, Employee_T AS M
WHERE E.EmployeeSupervisor = M.EmployeeID;
```

EMPLOYEEID	EMPLOYEE...	MANAGER
555955585	Mary Smith	Lawrence Haley
454-56-768	Robert Lewis	Phil Morris
Laura	Laura Ellenburg	Robert Lewis
123-44-345	Phil Morris	Robert Lewis
332445667	Lawrence Haley	Robert Lewis

5 rows total

Subqueries

- Placing an inner query (SELECT … FROM … WHERE) within a WHERE or HAVING clause of another query
- **Joining Technique:** when data from several relations are to be retrieved and displayed and the relationships are not necessarily nested;
- **Subquery Technique:** display data from only the tables mentioned in the outer query

Subqueries

- **Q27:** What are the name and address of the customer who placed order number 4?

Joining Technique:

```
SELECT CustomerName, CustomerAddress, CustomerCity, CustomerState,
CustomerPostalCode
FROM "db_pvfc11_big"."Customer_T", Order_T
WHERE Customer_T.CustomerID = Order_T.CustomerID
   AND OrderID = 4;
```

Subquery Technique:

```
SELECT CustomerName, CustomerAddress, CustomerCity, CustomerState,
CustomerPostalCode
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerID =
  (SELECT Order_T.CustomerID
   FROM "db_pvfc11_big"."Order_T"
   WHERE OrderID = 4);
```

Subqueries

- **Q28:** What are the names of customers who have placed orders?

```
SELECT CustomerName
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerID IN
  (SELECT DISTINCT CustomerID
   FROM "db_pvfc11_big"."Order_T");
```

CUSTOMERNAME
ABC Furniture Co.
Home Furnishings
Flanigan Furniture
A Carpet
Wild Bills
Dunkins Furniture
Ikards
Furniture Gallery
Contemporary Casuals
Janets Collection
Eastern Furniture

Subqueries

- The qualifiers NOT, ANY, and ALL may be used in front of IN or with logical operators such as =, >, and <.
- Also can use < ANY or >= ALL

- **Q29:** Which customers have not placed any orders for computer desk?

```
SELECT CustomerName
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerID NOT IN
  (SELECT DISTINCT CustomerID
   FROM "db_pvfc11_big". "Order_T", "db_pvfc11_big". "OrderLine_T", "db_pvfc11_big". "Product_T"
   WHERE Order_T.OrderID = OrderLine_T.OrderID
     AND OrderLine_T.ProductID = Product_T.ProductID
     AND ProductDescription = 'Computer Desk');
```

Subqueries

- Also can use ANY or ALL

- **Q30:** List the details about the product with the highest standard price.

```
SELECT ProductDescription, ProductFinish, ProductStandardPrice
FROM "db_pvfc11_big". "Product_T"
WHERE ProductStandardPrice >= ALL
  (SELECT ProductStandardPrice
   FROM "db_pvfc11_big". "Product_T");
```

EXISTS, NOT EXISTS

- EXISTS: True if the subquery returns an intermediate results table that is not empty; False if the subquery returns an empty table.
- NOT EXISTS: reverse of EXISTS

- **Q31:** What are the order IDs for all orders that have included furniture finished in Oak?

```
SELECT DISTINCT OrderID
FROM "db_pvfc11_big". "OrderLine_T"
WHERE EXISTS
  (SELECT *
   FROM "db_pvfc11_big"."Product_T"
   WHERE ProductID = OrderLine_T.ProductID
     AND ProductFinish = 'Oak');
```

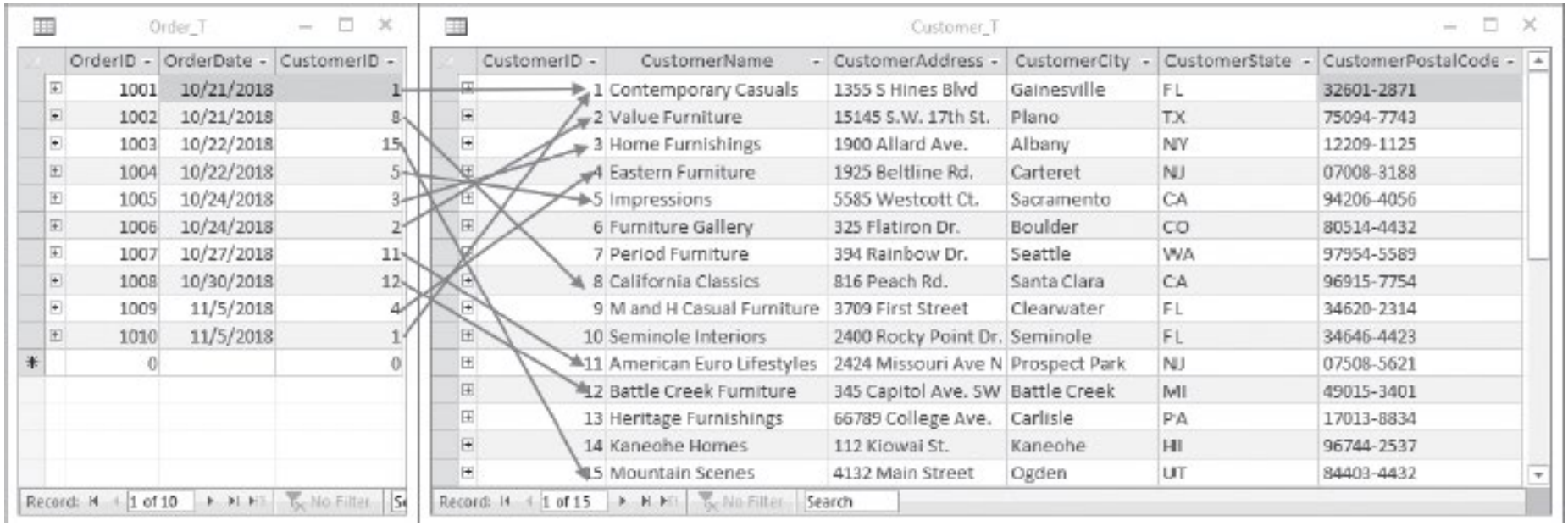
Clemson *Means*
BUSINESS

Chapter 4
Structural Query Language (SQL)
Part II. Joining Tables

Instructor: He Li

Queries for Multiple Tables

- **Join:** A relational operation that causes two or more tables with a common domain to be combined into a single table or view



Order_T

OrderID	OrderDate	CustomerID
1001	10/21/2018	1
1002	10/21/2018	8
1003	10/22/2018	15
1004	10/22/2018	5
1005	10/24/2018	3
1006	10/24/2018	2
1007	10/27/2018	11
1008	10/30/2018	12
1009	11/5/2018	4
1010	11/5/2018	1
0		0

Record: 1 of 10

Customer_T

CustomerID	CustomerName	CustomerAddress	CustomerCity	CustomerState	CustomerPostalCode
1	Contemporary Casuals	1355 S Hines Blvd	Gainesville	FL	32601-2871
2	Value Furniture	15145 S.W. 17th St.	Plano	TX	75094-7743
3	Home Furnishings	1900 Allard Ave.	Albany	NY	12209-1125
4	Eastern Furniture	1925 Beltline Rd.	Carteret	NJ	07008-3188
5	Impressions	5585 Westcott Ct.	Sacramento	CA	94206-4056
6	Furniture Gallery	325 Flatiron Dr.	Boulder	CO	80514-4432
7	Period Furniture	394 Rainbow Dr.	Seattle	WA	97954-5589
8	California Classics	816 Peach Rd.	Santa Clara	CA	96915-7754
9	M and H Casual Furniture	3709 First Street	Clearwater	FL	34620-2314
10	Seminole Interiors	2400 Rocky Point Dr.	Seminole	FL	34646-4423
11	American Euro Lifestyles	2424 Missouri Ave N	Prospect Park	NJ	07508-5621
12	Battle Creek Furniture	345 Capitol Ave. SW	Battle Creek	MI	49015-3401
13	Heritage Furnishings	66789 College Ave.	Carlisle	PA	17013-8834
14	Kaneohe Homes	112 Kiowai St.	Kaneohe	HI	96744-2537
15	Mountain Scenes	4132 Main Street	Ogden	UT	84403-4432

Record: 1 of 15 Search

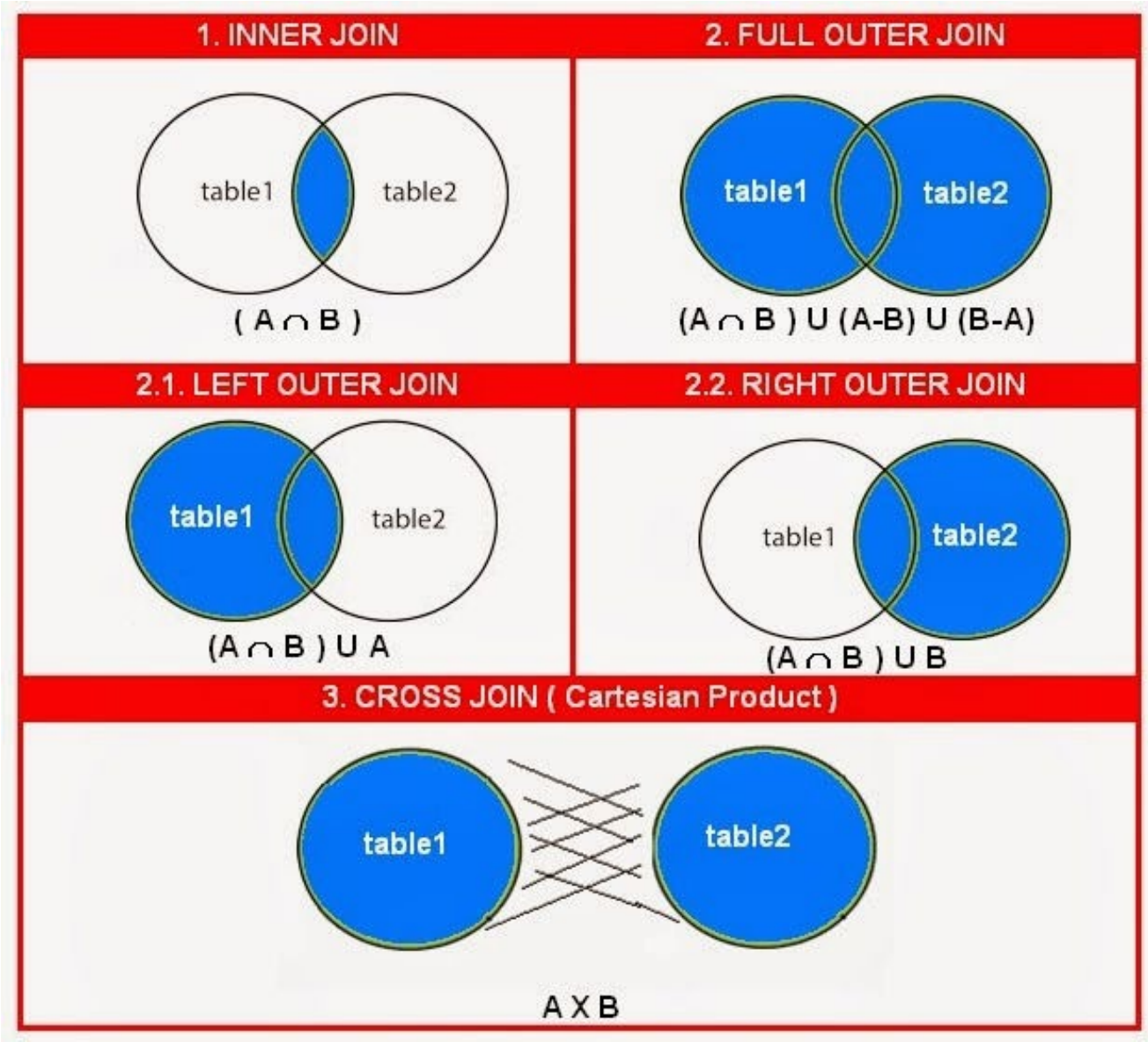
Types of Joins

Syntax: specify as **FROM** clause

table 1 explicit **JOIN** *table 2* **ON** joining condition

explicit: could be INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, or CROSS

With cross join, no need to specify condition



Inner Join

- **Q23:** What are the customer IDs and names of all customers, along with the order IDs for all the orders they have placed?

```
SELECT Customer_T.CustomerID, CustomerName, OrderID
FROM "db_pvfc11_big"."Customer_T" INNER JOIN Order_T
on Customer_T.CustomerID = Order_T.CustomerID;

SELECT Customer_T.CustomerID, CustomerName, OrderID
FROM "db_pvfc11_big"."Customer_T", Order_T
WHERE Customer_T.CustomerID = Order_T.CustomerID;
```

CUSTOME...	CUSTOME...	ORDERID
16	ABC Furniture C	51
3	Home Furnishing	2
12	Flanigan Furnitur	47
9	A Carpet	66
14	Wild Bills	58
8	Dunkins Furnitur	35
13	Ikards	59
6	Furniture Gallery	9
1	Contemporary C	46

Left Outer Join

- **Q24:** List customer name, identification number, and order number for all customers listed in the customer table. Include the customer identification number and name even if there is no order available for that customer.

```
SELECT Customer_T.CustomerID, CustomerName, OrderID
FROM "db_pvfc11_big"."Customer_T" LEFT OUTER JOIN Order_T
on Customer_T.CustomerID = Order_T.CustomerID;
```

CUSTOME...	CUSTOME...	ORDERID
16	ABC Furniture C	53
3	Home Furnishing	2
12	Flanigan Furnitur	45
9	A Carpet	66
5	Impressions	
14	Wild Bills	58
8	Dunkins Furniture	35
2	Value Furnitures	
13	Ikards	59
6	Furniture Gallery	44
1	Contemporary C	48
15	Janets Collection	34
4	Eastern Furniture	30
16	ABC Furniture C	55

Page 1 of 2 (61 rows total)

Right Outer Join

- **Q25:** List customer name, identification number, and order number for all customers listed in the customer table. Include the order number, even if there is no customer name and identification number available.

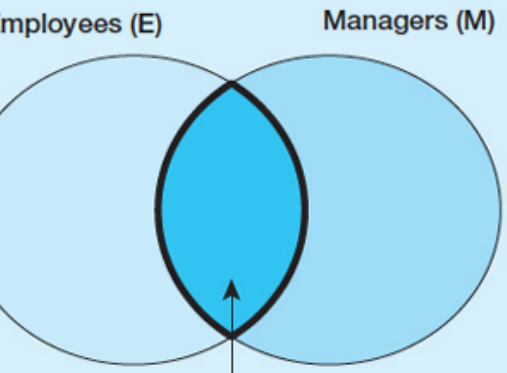
```
SELECT Customer_T.CustomerID, CustomerName, OrderID
FROM "db_pvfc11_big"."Customer_T" RIGHT OUTER JOIN Order_T
on Customer_T.CustomerID = Order_T.CustomerID;
```

CUSTOME...	CUSTOME...	ORDERID
16	ABC Furniture C	52
3	Home Furnishing	2
12	Flanigan Furnitur	45
9	A Carpet	56
14	Wild Bills	58
8	Dunkins Furnitur	50
13	Ikards	59
6	Furniture Gallery	44
1	Contemporary C	3
15	Janets Collectior	34
4	Eastern Furniture	49
16	ABC Furniture C	55
12	Flanigan Furnitur	47
9	A Carpet	66

Page 1 of 2 (58 rows total)

Self-Join

- A join requires matching rows in a table with other rows in that same table, e.g., unary relationship.



Employees who have supervisors; i.e.,
WHERE E.EmployeeSupervisor = M.EmployeeID

Employees (E)

EmployeeID	EmployeeName	EmployeeSupervisor
098-23-456	Sue Miller	
107-55-789	Stan Getz	
123-44-347	Jim Jason	678-44-546
547-33-243	Bill Blass	
678-44-546	Robert Lewis	

Managers (M)

EmployeeID	EmployeeName	EmployeeSupervisor
098-23-456	Sue Miller	
107-55-789	Stan Getz	
123-44-347	Jim Jason	678-44-546
547-33-243	Bill Blass	
678-44-546	Robert Lewis	

Self-Join

- **Q26:** What are the employee ID and name of each employee and the name of his or her supervisor?

```
SELECT E.EmployeeID, E.EmployeeName, M.EmployeeName AS Manager
FROM "db_pvfc11_big"."Employee_T" AS E, Employee_T AS M
WHERE E.EmployeeSupervisor = M.EmployeeID;
```

EMPLOYEEID	EMPLOYEE...	MANAGER
555955585	Mary Smith	Lawrence Haley
454-56-768	Robert Lewis	Phil Morris
Laura	Laura Ellenburg	Robert Lewis
123-44-345	Phil Morris	Robert Lewis
332445667	Lawrence Haley	Robert Lewis

5 rows total

Subqueries

- Placing an inner query (SELECT … FROM … WHERE) within a WHERE or HAVING clause of another query
- **Joining Technique:** when data from several relations are to be retrieved and displayed and the relationships are not necessarily nested;
- **Subquery Technique:** display data from only the tables mentioned in the outer query

Subqueries

- **Q27:** What are the name and address of the customer who placed order number 4?

Joining Technique:

```
SELECT CustomerName, CustomerAddress, CustomerCity, CustomerState,
CustomerPostalCode
FROM "db_pvfc11_big"."Customer_T", Order_T
WHERE Customer_T.CustomerID = Order_T.CustomerID
   AND OrderID = 4;
```

Subquery Technique:

```
SELECT CustomerName, CustomerAddress, CustomerCity, CustomerState,
CustomerPostalCode
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerID =
  (SELECT Order_T.CustomerID
   FROM "db_pvfc11_big"."Order_T"
   WHERE OrderID = 4);
```

Subqueries

- **Q28:** What are the names of customers who have placed orders?

```
SELECT CustomerName
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerID IN
  (SELECT DISTINCT CustomerID
   FROM "db_pvfc11_big"."Order_T");
```

CUSTOMERNAME
ABC Furniture Co.
Home Furnishings
Flanigan Furniture
A Carpet
Wild Bills
Dunkins Furniture
Ikards
Furniture Gallery
Contemporary Casuals
Janets Collection
Eastern Furniture

Subqueries

- The qualifiers NOT, ANY, and ALL may be used in front of IN or with logical operators such as =, >, and <.
- Also can use < ANY or >= ALL

- **Q29:** Which customers have not placed any orders for computer desk?

```
SELECT CustomerName
FROM "db_pvfc11_big"."Customer_T"
WHERE CustomerID NOT IN
  (SELECT DISTINCT CustomerID
   FROM "db_pvfc11_big". "Order_T", "db_pvfc11_big". "OrderLine_T", "db_pvfc11_big". "Product_T"
   WHERE Order_T.OrderID = OrderLine_T.OrderID
     AND OrderLine_T.ProductID = Product_T.ProductID
     AND ProductDescription = 'Computer Desk');
```

Subqueries

- Also can use ANY or ALL

- **Q30:** List the details about the product with the highest standard price.

```
SELECT ProductDescription, ProductFinish, ProductStandardPrice
FROM "db_pvfc11_big". "Product_T"
WHERE ProductStandardPrice >= ALL
  (SELECT ProductStandardPrice
   FROM "db_pvfc11_big". "Product_T");
```

EXISTS, NOT EXISTS

- EXISTS: True if the subquery returns an intermediate results table that is not empty; False if the subquery returns an empty table.
- NOT EXISTS: reverse of EXISTS

- **Q31:** What are the order IDs for all orders that have included furniture finished in Oak?

```
SELECT DISTINCT OrderID
FROM "db_pvfc11_big". "OrderLine_T"
WHERE EXISTS
  (SELECT *
   FROM "db_pvfc11_big"."Product_T"
   WHERE ProductID = OrderLine_T.ProductID
     AND ProductFinish = 'Oak');
```